Deadlock

1. What is Deadlock?

In a **multi-programming system**, many processes run together and fight for limited resources like CPU, memory, I/O devices, files, locks, semaphores, sockets etc.

A **deadlock (DL)** happens when:

- Two or more processes are **waiting forever** for resources which are **held by each other**.
- No process can move ahead.
- They remain stuck permanently.

Real-World Analogy

Two people on a narrow bridge:

- Person A waits for B to move.
- Person B waits for A to move.
- No one moves → both stuck forever.

That is **deadlock**.

2. How a Process Uses a Resource?

Every process uses a resource in three steps:

1. **Request**
   - "Is the resource free? If yes, give it to me."
   - If not free → process waits.
2. **Use**
   - Process performs its work.
3. **Release**
   - Process frees the resource for others.

Real Life Example

A **printer** in a college lab:

- A student requests the printer.
- Prints the document.
- Releases the printer for the next student.

3. Necessary Conditions for Deadlock (All 4 Must Hold Together)

Deadlock occurs **only** when all four conditions are true at the same time.

a) Mutual Exclusion

A resource that **cannot be shared** by more than one process at a time.

Example

- Only one process can use a **printer** at a time.
- Same for a **lock** or **database write operation**.

b) Hold and Wait

A process is:

- **Holding** one resource
- **Waiting** for additional resources

Example

- Process P1 has **Printer**
- It wants **Scanner**
- Scanner is held by P2
- So P1 waits.

c) No Preemption

Resources **cannot be forcibly taken** from a process.

Example

You can't pull a printer away from someone in the middle of printing.

d) Circular Wait

A cycle such as:

P1 → waiting for R1 (held by P2)
P2 → waiting for R2 (held by P3)
P3 → waiting for R3 (held by P1)

Circle formed → **Deadlock**

Real Example

- Printer → held by P1
- Scanner → held by P2
- Camera → held by P3

P1 wants Scanner
P2 wants Camera
P3 wants Printer

Full circle → deadlock.

4. Real-World Example

Example: Two resources: Forks (Dining Philosopher Problem)

Five people sitting around a table. Each needs **two forks** to eat.

- Philosopher A picks left fork.
- Philosopher B picks left fork.
- Each waits for right fork.
- No one releases the first fork.

They all starve → **deadlock**

5. Deadlock Prevention

To **prevent** deadlock, the system ensures at least **one** of the four necessary conditions never holds.

A) Break Mutual Exclusion

Not possible for many resources (printer, lock, database write).

So this condition is usually **not broken**.

B) Break Hold & Wait

Two common methods:

Protocol A: Request All Resources at Once

Process must take **all** the resources it will ever need **before starting**.

Example

Before starting a printing + scanning job:

- Process must take **printer + scanner** together.

Problem:
Wastes resources because they remain idle even if not used immediately.

Protocol B: Allow Requests Only When Holding None

If a process wants a new resource:

- It must first release everything it holds.

Example

P1 holds printer, wants scanner
→ P1 must release printer
→ Then request scanner again
→ Then printer again.

C) Break No Preemption

If a process wants a resource that is unavailable:

System can **take away** its currently held resources.

Example

P1 has Printer
P1 also wants Scanner (busy)

System does:

- Take Printer from P1
- Give Scanner + Printer later together

Problem:

- Can cause **livelock** (process goes in loop of losing and regaining resources).

D) Break Circular Wait (Most Practical Method)

Impose a **global order** on resources.

Example:
Number all resources:

R1 < R2 < R3 < R4

Rule:

- A process can request resources **only in increasing order**.

Real Example

P1 wants R1 and R2 → must take R1 first
P2 wants R2 and R3 → must take R2 first

Because everyone requests in the same order, **circular wait cannot occur**.

This is used in:

- Database systems
- OS locks
- File locking

6. Deadlock Avoidance (Banker's Algorithm)

Avoidance means:

- System checks **future possibility**
- Only grants resources if it keeps system in a **safe state**

Real-World Analogy

Like a **bank**, OS checks:

- If giving a loan (resource) will keep the bank safe.
- If future withdrawals can be handled.

If not safe → request is denied.

7. Deadlock Detection & Recovery

Allow deadlock to happen → detect → fix.

A) Detection

OS periodically checks for:

- **Cycles** in resource allocation graph.

If cycle exists → deadlock exists.

B) Recovery Methods

1. Process Termination

Kill one or more processes to break the cycle.

2. Resource Preemption

Take resources away from some process.

3. Rollback

Restart process from last checkpoint.

8. Deadlock Ignorance (Ostrich Algorithm)

- OS ignores deadlocks.
- Used in **Windows**, **Linux**.
- Because deadlocks are rare.
- Handling them costs more than ignoring them.

Like an ostrich putting its head in sand and pretending nothing happened.

Real-Life Examples of Deadlock

Example 1: Traffic Deadlock

Cars at a 4-way crossing:

- Each car blocks the next.
- No one can move.
- Perfect circular wait.

Example 2: Database Deadlock

Transaction T1: updates Row A → wants Row B
Transaction T2: updates Row B → wants Row A

Summary

Concept	Meaning
Deadlock	Processes stuck forever
4 Conditions	Mutual Exclusion, Hold&Wait, No Preemption, Circular Wait
Prevention	Break one condition
Avoidance	Banker's algorithm
Detection	Check cycles
Recovery	Kill process / preemption
Ignorance	Pretend deadlocks don't exist